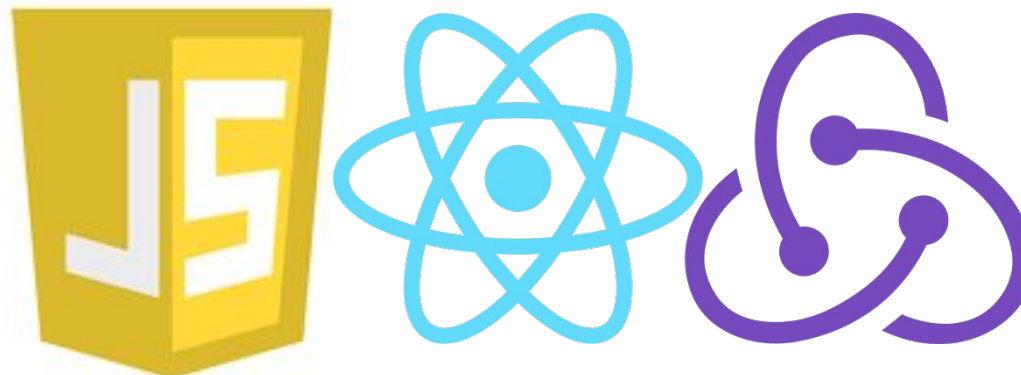


Лекція 15

Сучасний фронтенд

Базова архітектура сайту



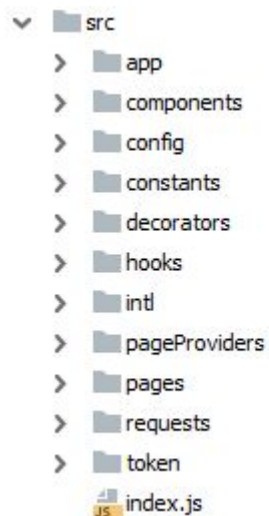
1. Основні компоненти.

При проектуванні сайту можна виділити наступні основні компоненти:

- Files/Directories hierarchy (структура та ієрархія файлів та папок)
- Routing (роутинг)
- Internationalization (i18n - інтернаціоналізація)
- Requests (запити)
- Authorization (авторизація)
- Authorities (доступність функціоналу за наявності певних привілеїв користувача)

2. Структура проекту.

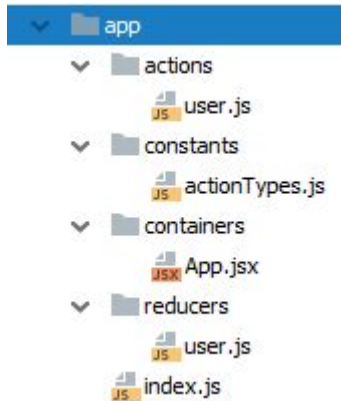
Пропонується наступна структура папок:



де в **index.js** файлі здійснюється вбудовування реакт-проекту в основний HTML код сайту.

2.1. Структура проекту. Каталог app.

У папці **app** розміщується основний контейнер програми.

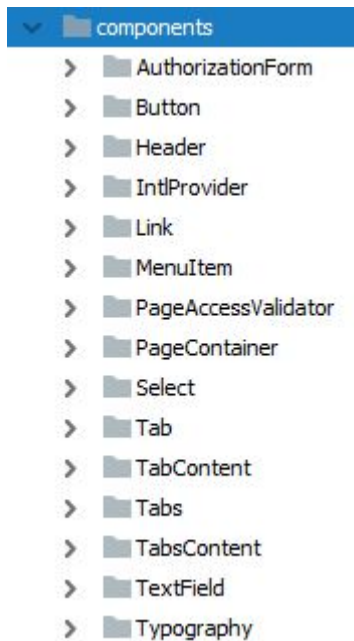


У цьому контейнері здійснюється підключення всіх допоміжних провайдерів, таких як:

- **redux store** додатка (тут же зберігається логіка загальних для всього додатка ред'юсерів)
- **intl provider** (служить для інтернаціоналізації програми)
- **routing** (служить для навігації по сторінках додатку)

2.2. Структура проекту. Каталог components.

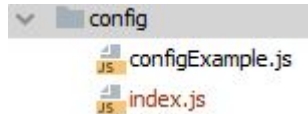
У папці **components** розміщуються всі компоненти, що використовуються в проекті (кнопки, списки, хедер і т.д.)



Навіть якщо у проекті використовуються готові компоненти із сторонньої бібліотеки (наприклад з material-ui), то кожен компонент повинен бути поміщений в обгортку, в якій повинні бути оголошені всі можливі props, що передаються компоненту, при його використанні.

2.3. Структура проекту. Каталог config.

У папці **config** зазвичай розміщується 2 файли:



У файлі **configExample.js** міститься об'єкт, у якому поля - це можуть бути імена мікросервісів (з якими спілкується UI), або імена якихось секретних ключів. У цьому файлі значення всіх полів порожні і він може бути вільно закоммічений у репозитії.

```
export default {  
  BASE_URL: '',  
  USERS_SERVICE: '',  
  ANY_SERVICE1: '',  
  ANY_SERVICE2: '',  
  someSecretKey: '',  
}
```

У файлі **index.js** міститься об'єкт із тими самими ключами, що у **configExample.js**, але з конкретними значеннями. Цей файл повинен бути доданий до **.gitignore**

2.4. Структура проекту. Каталог constants.

У папці **constants** зазвичай розміщуються файли глобальних констант:



У нашому випадку використовуються всього 2 набори констант:

- **languages** (зберігає доступні для нашої програми коди мов)

```
export const en = 'en';  
export const ru = 'ru';  
export const ua = 'ua';
```

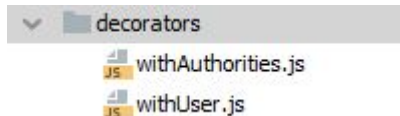
- **pages** (зберігає список усіх сторінок, які є в нашому додатку)

```
export const INITIAL = 'initial';  
export const LOGIN = 'login';
```

Сюди також можна додати, наприклад, константи кольорів, які будуть використані для стилізації нашої програми.

2.5. Структура проекту. Каталог decorators.

У папці **decorators** розташовуються функції-декоратори:



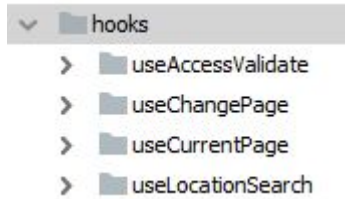
Такі функції приймають аргументом реакт-компонент, і повертають його, але додаючи в нього props. Це потрібно для тих випадків, коли функція-декоратор має доступ до якоїсь сутності (наприклад до store додатка), а компонент, що передається в аргументі - вже не матиме доступу до цієї сутності (наприклад, до store додатка, тому що цей компонент створить свій власний store). У такому разі функція-декоратор зможе звернутися до store додатка, взяти у нього потрібне компоненті поле з редьюсера, і передати його

В пропси компоненти:

```
import { useSelector } from 'react-redux';
import React from 'react';
const withAuthorities = Component => (props) => {
  const authorities = useSelector(({ user }) => user.authorities);
  return (
    <Component
      authorities={authorities}
      {...props}
    />
  );
};
export default withAuthorities;
```


2.6. Структура проекту. Каталог hooks.

У папці **hooks** розташовуються самописні хуки:

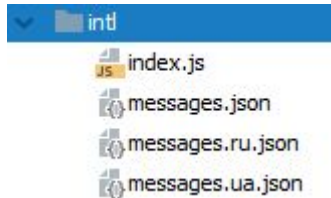


У цьому прикладі є 4 хуки:

- **useAccessValidate** — приймає об'єкт з полями **ownedAuthorities** (масив) і **neededAuthorities** (масив), перевіряє чи містить **ownedAuthorities** всі елементи з **neededAuthorities**, і повертає значення true чи false. Цей хук потрібен для визначення, чи має користувач доступ до будь-якого функціоналу.
- **useChangePage** — повертає функцію, яка приймає об'єкт з полями **locationSearch** і **path**. Поле **locationSearch** - це об'єкт, з полями, що відповідають параметрам з query поточного URL. Поле **path** - це рядок, який відповідає URL нової сторінки.
- **useCurrentPage** — повертає строку, що відповідає значенню однієї з констант із файлу **constants/pages.js**.
- **useLocationSearch** — усуває невалідні значення параметрів із URL query (замінює їх на валідні дефолтні значення), і повертає об'єкт із валідними полями відповідними параметрами із query поточного URL.

2.7. Структура проекту. Каталог intl.

У папці **intl** розміщуються файли інтернаціоналізації:



У кожному **.json** файлі лежать повідомлення (у форматі "ключ": "значення"), що відповідають мові, вказаній в імені файлу. Файл **messages.json** є дефолтним файлом і відповідає англійській мові.

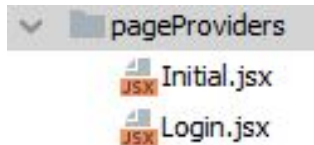
У файлі **index.js** експортується функція, яка приймає на вхід рядок **lang** (поточна мова інтерфейсу), і динамічно завантажує JSON-об'єкт повідомлень з відповідної мови файлу, і повертає цей об'єкт.

Функція з **intl/index.js** імпортується і викликається у компоненті **components/IntlProvider**, де вхідний параметр **lang** дістається з query URL браузера за допомогою хука **hooks/useLocationSearch**:

```
...  
const {  
  lang,  
} = useLocationSearch();  
const messages = useMemo(() => getMessages(lang), [lang]);  
...
```

2.8. Структура проекту. Каталог page Providers.

У папці **pageProviders** розташовуються компоненти, які виступають як би обгортками над кожною сторінкою, яка є в проекті:

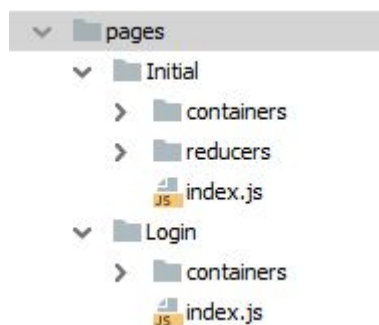


Такі обгортки є додатковим ступенем свободи, і в них можна розмістити код, який, наприклад, відповідає за доступність сторінки за наявністю привілеїв, або код, який регулює ширину сторінки і т.д.:

```
import React from 'react';
import PageAccessValidator from 'components/PageAccessValidator';
import InitialPage from 'pages/Initial';
import PageContainer from 'components/PageContainer';
const Initial = () => (
  <PageAccessValidator>
    <PageContainer>
      <InitialPage />
    </PageContainer>
  </PageAccessValidator>
);
export default Initial;
```

2.9. Структура проекту. Каталог pages.

У папці **pages** розміщуються компоненти-сторінки, які вже мають свої ред'юсери, і які не мають доступу до зовнішніх ред'юсерів. Кожна сторінка самостійно займається завантаженням та відображенням своїх даних. Єдиний (правильний) спосіб передати інформацію з однієї сторінки на іншу - це URL query



2.10. Структура проекту. Каталог requests.

У папці **requests**, в єдиному файлі **index.js**, розташовуються функції обгортки для здійснення rest-запитів на сторонні сервіси (наприклад **getJSON** і **postJson**).



Ці функції – це обгортки над стандартною функцією **fetch**. Такі обгортки, додають до запитів **headers** (щоб щоразу не дублювати цей шматок коду), і обробляють відповідь, відразу виділяючи дані у форматі json (якщо відповідь передбачає json формат).

Ще одне важливе завдання, яке виконують ці обгортки - це додавання токена авторизації (який дістається з **localStorage** браузера) в headers кожного запиту:

```
import { getToken } from 'token';  
const getHeaders = () => ({  
  Accept: 'application/json',  
  Authorization: `Bearer ${getToken()}`,  
  'Content-Type': 'application/json',  
});
```

3. Список корисних посилань.

Для кращого ознайомлення з **роутингом** рекомендовано ознайомитися з прикладами цього ресурсу:

<https://reactrouter.com/en/main/start/tutorial>

Для кращого розуміння, що таке **localStorage** рекомендовано ознайомитися з

<https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>